

Chapter 1: Variables, Data Types, & Type Casting (1–50)

1. Assign your name to a variable and print it.
 2. Create three variables for day, month, and year, then print them formatted as DD/MM/YYYY.
 3. Take an integer input from a user and print its data type.
 4. Take a float input, cast it to an integer, and print the result.
 5. Create a program that inputs a user's first and last name and prints them in reverse order with a space.
 6. Swap the values of two variables using a temporary variable.
 7. Swap the values of two variables *without* using a temporary variable.
 8. Take a string input containing a number (e.g., "123") and convert it to an integer.
 9. Take an integer input and convert it to a float.
 10. Input a single character and print its boolean value using `bool()`.
 11. Initialize 5 different variables on a single line of code.
 12. Assign the same value to 3 different variables simultaneously on one line.
 13. Input a sentence and print its total length using `len()`.
 14. Create a program that takes a price as a float and prints it formatted with a currency symbol (e.g., \$19.99).
 15. Set a variable to `None` and print its type.
 16. Input two strings and concatenate them with a space in between.
 17. Input a string and an integer, then print the string repeated that integer number of times.
 18. Ask a user for their birth year and calculate their age in the year 2026.
 19. Take a multi-line string input using triple quotes and print it exactly as typed.
 20. Create a variable with a valid name and another with an invalid name (commented out) to demonstrate naming rules.
 21. Take a user's weight in kilograms (float) and convert it to a string ending with " kg".
 22. Take two integer inputs on a single line using `split()`.
 23. Declare a boolean variable and print its inverse using the `not` operator.
 24. Input a number and print its value multiplied by itself using a single variable assignment.
 25. Read a user's favorite color and print "Your favorite color is [color]".
 26. Take a string input and check if its length is greater than 10 (print `True` or `False`).
 27. Cast the boolean values `True` and `False` into integers and print them.
 28. Create a variable tracking a bank balance as a float, add 100 to it, and print the updated value.
 29. Take an input and display it wrapped in brackets like `[input]`.
 30. Create a program that reads a user's middle name; if they hit enter without typing, store it as an empty string.
 31. Print the value of an integer variable left-padded with zeros to a width of 5 digits (e.g., 00042).
 32. Define a constant-like variable for gravity (\$9.81\$) and print it with a descriptive message.
 33. Create a program that receives a sentence, casts its length to a float, and outputs it.
 34. Ask for a username, strip any leading or trailing whitespaces from it, and save it.
-

35. Declare a variable containing a large number using underscores for readability (e.g., `1_000_000`) and print it to see how Python treats it.
36. Ask for a user's phone number as an input and store it strictly as a string data type.
37. Input two float numbers, add them, and round the final result to 2 decimal places.
38. Take a boolean input by casting a string input directly, and explain why typing `"False"` still yields `True`.
39. Define a variable using a legal name containing an underscore, a number, and letters.
40. Take an input and print it 10 times separated by tabs (`\t`).
41. Set a variable to a string containing a newline character (`\n`) and print it to show the split line.
42. Input a number as a string, convert it to a float, then convert that float to an integer.
43. Create a profile generator that inputs name, job, and city, then prints a short bio sentence.
44. Input a distance in kilometers, convert it to an integer, and print it.
45. Print a string that contains literal double quotes inside it (e.g., `He said, "Hello"`).
46. Create a variable holding a negative integer and print its absolute value using `abs()`.
47. Input a number, add 5 to it using the `+=` operator, and print it.
48. Take an input and print its memory address using `id()`.
49. Create a program that requests a percentage value (e.g., 85) and outputs it divided by 100 as a true float decimal.
50. Print the text `"Python Basic Mastery"` centered within a line of 40 characters using formatting tools.

Chapter 2: Operators & Arithmetic Expressions (51–100)

51. Input two numbers and print their sum.
52. Input two numbers and subtract the second from the first.
53. Multiply two float numbers together and print the result.
54. Divide two numbers and print the precise float result.
55. Perform floor division (`//`) on two numbers and print the integer result.
56. Find the remainder when dividing an input integer by another using the modulus (`%`) operator.
57. Calculate the result of raising a base number to an exponent power (`$x^y$`).
58. Ask a user for a number of seconds and convert it into minutes and remaining seconds.
59. Input a total number of days and convert it into weeks and remaining days.
60. Find the area of a rectangle given its width and height inputs.
61. Find the perimeter of a rectangle given its width and height inputs.
62. Compute the area of a circle given its radius using `$3.14159$`.
63. Calculate the perimeter (circumference) of a circle given its radius.
64. Convert a temperature from Celsius to Fahrenheit using the formula: `$F = (C \times 9/5) + 32$`.
65. Convert the temperature from Fahrenheit to Celsius.
66. Compute the average of four test scores input by the user.
67. Calculate simple interest given principal, rate of interest, and time inputs (`$SI = \frac{P \times R \times T}{100}$`).

68. Find the volume of a cube given the length of one side.
69. Take a 2-digit number from the user and print the sum of its digits using arithmetic operators only.
70. Take a 3-digit number from the user and isolate the middle digit using floor division and modulus.
71. Evaluate the mathematical expression $a^2 + b^2$ using inputs a and b .
72. Demonstrate operator precedence by writing an expression that performs multiplication before addition.
73. Force addition to occur before multiplication in an expression using parentheses.
74. Input a price and a tax percentage rate, then calculate the total final price including tax.
75. Create a program that divides a restaurant bill evenly among a specified number of friends.
76. Input a distance in miles and convert it to kilometers ($1 \text{ mile} \approx 1.60934 \text{ km}$).
77. Input a weight in pounds and convert it to kilograms ($1 \text{ lb} \approx 0.453592 \text{ kg}$).
78. Calculate a car's fuel efficiency in miles per gallon given miles driven and gallons used.
79. Calculate the total cost of a purchase given item price, quantity, and a flat discount amount subtracted at the end.
80. Input a number and check if it is divisible by 5 using the modulus operator (print 0 for true, non-zero for false).
81. Use the $+=$ operator to increment a counter variable by 10.
82. Use the $-=$ operator to decrement a counter variable by 3.
83. Use the $*=$ operator to triple the value of an input number.
84. Use the $/=$ operator to halve the value of an input number.
85. Input a total number of items and a box capacity, then calculate how many full boxes you can pack.
86. Using the same items and box capacity as above, calculate how many items will be left over.
87. Find the total monthly salary of an employee given their hourly wage and total hours worked in a month.
88. Compute the square and the cube of an input number in a single print statement.
89. Calculate the body mass index (BMI) using the formula: $\text{BMI} = \frac{\text{weight in kg}}{(\text{height in meters})^2}$.
90. Input a number of inches and convert it to centimeters ($1 \text{ inch} = 2.54 \text{ cm}$).
91. Calculate the sale price of an item given its original price and a discount percentage (e.g., \$20% off).
92. Input the length of a square's side and calculate its diagonal length using the formula $\text{side} \times 1.414$.
93. Find the total cost of electricity given the number of units consumed and the cost per unit.
94. Take a 4-digit year (e.g., 2026) and extract the last two digits using an arithmetic operator.
95. Compute the net salary of an employee after deducting a flat 12% tax from their gross earnings.
96. Find the area of a triangle using the standard formula: $\frac{1}{2} \times \text{base} \times \text{height}$.
97. Determine the total minutes elapsed given an input of hours, minutes, and seconds.
98. Find the quotient and remainder of two numbers simultaneously using separate operators.
99. Input an amount in cents (e.g., 345) and break it down into whole dollars and remaining cents.

100. Write a program that takes an integer, multiplies it by 2, adds 10, and divides the result by 2 to demonstrate a math trick.

Chapter 3: Conditional Statements (101–150)

101. Input a number and print "Positive" if it is greater than zero.
102. Input a number and print "Negative" if it is less than zero.
103. Input a number and print "Zero" if it equals zero.
104. Check if an input number is even or odd.
105. Input an age and print "Eligible to Vote" if the age is 18 or older, otherwise print "Not Eligible".
106. Take two numbers and print the larger one.
107. Take two numbers and print the smaller one.
108. Input three numbers and find the maximum among them using nested if statements.
109. Input three numbers and find the minimum among them using elif statements.
110. Determine if a given year input by the user is a leap year or a common year.
111. Input a student's score (0–100) and output their letter grade (A, B, C, D, F).
112. Check if an input character is a lowercase letter, uppercase letter, or a number.
113. Input a single alphabet character and check if it is a vowel or a consonant.
114. Check if a number is divisible by both 3 and 5 at the same time.
115. Check if a number is divisible by 3 or divisible by 5, but not necessarily both.
116. Ask a user for a password input; print "Access Granted" if it matches a secret password string, otherwise "Access Denied".
117. Input a person's height in cm. If it's less than 150 cm print "Short", 150-180 cm print "Average", above 180 cm print "Tall".
118. Create a simple 4-option calculator (+, -, *, /) using if-elif-else based on user input selection.
119. Input a temperature. If it's below 0 print "Freezing", 0 to 20 print "Cold", 21 to 35 print "Warm", above 35 print "Hot".
120. Check if a given number falls within a specific range (e.g., between 10 and 50 inclusive).
121. Input a month number (1–12) and print the corresponding month name.
122. Input a month number (1–12) and print the total number of days in that month (assume a non-leap year).
123. Input a weekday number (1–7) and print whether it is a "Weekday" or a "Weekend".
124. Check if a string input is completely empty or contains text.
125. Ask a user for two numbers and an operator. If the operator is division and the second number is 0, print an error message instead of dividing.
126. Create a system that checks an account balance and an explicit withdrawal request amount. Print "Insolvent" if the withdrawal exceeds the balance.
127. Check if a 3-digit number input is an Armstrong number (sum of the cubes of its digits equals the number itself).

128. Input a triangle's three angles and verify if they form a valid triangle (sum must equal 180 degrees).
129. Input the lengths of three sides of a triangle and determine if it is equilateral, isosceles, or scalene.
130. Check if a number is a multiple of 10.
131. Create a login validation script checking both a username and a password variable.
132. Input an integer and check if it is a perfect square (its square root has no fractional part).
133. Check if a given string contains a space character inside it.
134. Evaluate a user's age to calculate a movie ticket price: \$5 for children under 12, \$12 for adults, \$8 for seniors ($65 + \$$).
135. Check if an input integer is a 2-digit, 3-digit, or multi-digit number.
136. Input two strings and check if they are identical in value but ignore case sensitivity (e.g., "Python" equals "python").
137. Check if a given number is positive and even at the same time.
138. Check if a given number is negative and odd at the same time.
139. Ask for an input speed. If it exceeds 60, print "Speeding Ticket", otherwise print "Safe Driving".
140. Create a basic rock-paper-scissors comparison step: input Player 1 choice and Player 2 choice, then print who wins.
141. Determine if a given number is closer to 0 or closer to 100.
142. Input a character and check if it is a special punctuation symbol (e.g., !, @, #, \$).
143. Check if a user's income qualifies them for a tax bracket: under \$20,000 pays 0%, over \$20,000 pays 10%.
144. Input a text string and check if it starts with the letter "A".
145. Input a text string and check if it ends with the letter "Z".
146. Input a number and verify if it is a single-digit number (between -9 and 9).
147. Check if a package weight exceeds 20 kg. If it does, calculate an extra surcharge fee of \$5 per kg over the limit.
148. Input an hour in 24-hour format (0–23). Print "Morning", "Afternoon", "Evening", or "Night".
149. Check if a student passed both internal exam (minimum 40) and external exam (minimum 40) to pass overall.
150. Create an inline ternary conditional assignment that sets a variable to "Pass" if a score is 50 or higher, else "Fail".

Chapter 4: While Loops & Basic Iteration (151–200)

151. Print numbers from 1 to 10 using a while loop.
 152. Print numbers from 10 down to 1 using a while loop.
 153. Print all even numbers between 1 and 20 using a while loop.
 154. Print all odd numbers between 1 and 20 using a while loop.
-

155. Calculate the sum of the first 10 natural numbers using a `while` loop.
156. Keep asking the user to input a number until they input a negative number.
157. Create an infinite `while` loop and explicitly break out of it when a counter hits 5.
158. Write a program that reads numbers from a user until they type 0, then prints the running total sum.
159. Count the number of digits in an input integer using a `while` loop and integer division.
160. Print the multiplication table of a given number using a `while` loop.
161. Find the sum of all digits of an input number using a `while` loop.
162. Reverse an integer number using mathematical operations inside a `while` loop.
163. Print the first 10 multiples of 7 using a `while` loop.
164. Create a countdown timer simulation that starts at an input number and prints numbers down to 0.
165. Read a series of integers from the user until they type "stop", then print the average of those numbers.
166. Print a string character by character using a `while` loop and an indexing pointer variable.
167. Use a `while` loop to find the product of numbers from 1 to 5 (factorial of 5).
168. Keep asking a user to guess a hidden password; stop only when they guess correctly.
169. Use a `while` loop to print numbers from 1 to 30, skipping any numbers that are multiples of 5 using `continue`.
170. Find the largest number from a series of positive numbers entered by a user, ending the input loop when -1 is entered.
171. Print the squares of numbers from 1 to 10 using a `while` loop.
172. Print a sequence of numbers starting from 1 where each number doubles until it exceeds 1000.
173. Print all numbers between 100 and 200 that are divisible by 11 using a `while` loop.
174. Count how many odd numbers exist between two numbers input by the user.
175. Create a menu-driven program that loops until the user selects option "Quit".
176. Keep taking names from the user until they enter an empty string, then print total names entered.
177. Use a `while` loop to clear or decrement a variable from 100 to 0 in steps of 5.
178. Repeatedly divide a number by 2 using a `while` loop until the number becomes less than 1, tracking the step count.
179. Print the character code (`ord()`) of characters in a string dynamically until the string sequence ends.
180. Print a horizontal line of 20 dashes using a `while` loop counter.
181. Accumulate a string by repeatedly asking the user for words and appending them until they enter ".".
182. Print all integers between -5 and $+5$ using a `while` loop.
183. Check if an input number is prime using a `while` loop to test divisors from 2 up to the number minus one.
184. Use a `while` loop to repeatedly prompt for a valid percentage (0–100); keep looping if the input falls outside this range.

185. Print the cube of all odd integers from 1 to 15.
186. Count how many times a user enters an even number before entering their first odd number.
187. Simulate a basic ATM loop: present options to check balance, deposit, or withdraw; loop until exit.
188. Calculate the sum of all even numbers between two user-defined limits using a `while` loop.
189. Print the first N numbers in the Fibonacci sequence ($0, 1, 1, 2, 3, 5\ldots$) using a basic loop state swap.
190. Extract and print each individual digit of a number on a new line starting from the rightmost digit.
191. Print a grid of numbers where the inner loop prints columns up to a specified maximum using a `while` loop pattern.
192. Create a basic step counter that counts up to 50, but uses an inner flag to simulate pausing or skipping ranges.
193. Keep rolling a simulated digital die value (1-6) inside a loop until a 6 is rolled.
194. Find the smallest divisor of a number (greater than 1) using a `while` loop.
195. Accumulate integers starting from 1 until their combined sum crosses 100; print the final count of numbers used.
196. Take a string input and build a new string that drops all vowel characters using a character index tracking loop.
197. Use a `while` loop to check if a list-like string contains any numerical digits.
198. Print the phrase "Loading..." 5 times with a decrementing loop indicator.
199. Write a program that validates an explicit pin number entry, locking out after exactly 3 failed loop iterations.
200. Implement a `while-else` structure that prints a success message if the loop completes without hitting a `break`.

Chapter 5: For Loops & Range Sequences (201–250)

201. Iterate and print numbers from 1 to 10 using `for` and `range()`.
 202. Print numbers from 20 down to 10 using `range()`.
 203. Print all multiples of 5 from 0 to 50 using `range()`.
 204. Calculate the sum of all numbers from 1 to 100 using a `for` loop.
 205. Print the multiplication table of a number up to 12 using a `for` loop.
 206. Iterate through a string variable and print each character on a separate line.
 207. Count the total number of vowels present inside a string using a `for` loop and conditional check.
 208. Print all even numbers from 2 to 50 using a stepped `range()` function.
 209. Print all odd numbers from 99 down to 1 using a negative step `range()`.
 210. Compute the factorial of an input number using a `for` loop.
-

211. Print the squares of the first 15 integers using a loop.
212. Count how many times a specific character appears in an input sentence.
213. Print the FizzBuzz pattern from 1 to 30: print "Fizz" for multiples of 3, "Buzz" for 5, and "FizzBuzz" for both.
214. Given a hardcoded range of numbers, calculate how many are positive and how many are negative.
215. Print a horizontal line of 15 asterisks using a for loop.
216. Output numbers from 1 to 20, but use break to terminate the loop early if 13 is encountered.
217. Output numbers from 1 to 10, but use continue to skip 5.
218. Print a 3x3 square grid of hashtags (#) using nested for loops.
219. Print a right-angled triangle pattern of asterisks with a height of 5 lines using nested loops.
220. Compute the sum of all odd numbers between 1 and 50 using a for loop.
221. Given a string, print only the characters located at even index positions.
222. Create a table format displaying numbers from 1 to 5 alongside their squares and cubes.
223. Count the number of spaces present in a user's text input.
224. Sum all integers between two input bounds X and Y inclusive using `range(X, Y+1)`.
225. Take a number input and check if it is prime by testing factors inside a for loop.
226. Print every character in a string except for lowercase 'x'.
227. Convert a string of numbers (e.g., "12345") into an actual running sum of digits using a for loop.
228. Print an inverted right-angled triangle of asterisks (starting wide and getting narrower).
229. Count the total number of uppercase letters in an input string.
230. Count the total number of lowercase letters in an input string.
231. Print numbers from 1 to 100, but only those that are perfectly divisible by 7.
232. Print a checkerboard pattern of 0s and 1s using a nested loop.
233. Find the average value of a sequence of numbers defined within a `range()`.
234. Look through numbers from 1 to 50 and print only those whose last digit is 3.
235. Take a string and create a new string where every character is doubled (e.g., "cat" becomes "ccaatt").
236. Multiply all numbers from 1 to 10 together except for the number 0 or 5.
237. Print the alphabet characters implicitly by looping through ASCII code values using `chr()`.
238. Print the numbers 1 to 5 on a single line separated by commas, avoiding a trailing comma at the end.
239. Print a pyramid pattern of numbers where row 1 contains 1, row 2 contains 2 2, etc.
240. Calculate the sum of the series $1 + 2 + 3 + \dots + N$, where N is input by the user.
241. Count how many alphanumeric digits exist in a string using `.isalnum()` inside a loop.
242. Print the index number alongside each character of a string without using `enumerate()` (use a manual counter).
243. Print a countdown pattern that reads: "5 seconds left", "4 seconds left", down to "0".
244. Create a running product of all numbers within a specific range that are multiples of 3.

- 245. Print a square box border made of asterisks with an empty center using loops and boundaries.
- 246. Calculate the sum of squares of numbers from 1 to N^2 .
- 247. Run a loop from 1 to 10 and use a pass statement inside an if block for an empty branch demo.
- 248. Look through a range of years (e.g., 2000 to 2030) and print all leap years found.
- 249. Print every third word from a sentence by tracking space counts or loops.
- 250. Demonstrate a for-else loop that prints an alert if a target loop item is not found.

Chapter 6: Basic String Operations & Manipulations (251–300)

- 251. Access and print the first character of an input string.
- 252. Access and print the last character of a string using a negative index.
- 253. Extract and print a substring from index 2 to index 6 of a given string.
- 254. Reverse a string entirely using string slicing (`[::-1]`).
- 255. Convert an entire string input into uppercase letters.
- 256. Convert an entire string input into lowercase letters.
- 257. Capitalize only the very first letter of a sentence using a string method.
- 258. Convert a string to title case (capitalize the first letter of every word).
- 259. Check if a string contains only numeric digits using `.isdigit()`.
- 260. Check if a string contains only alphabetic letters using `.isalpha()`.
- 261. Swap the case of a string (uppercase becomes lowercase and vice versa).
- 262. Remove all leading and trailing whitespace characters from a string using `.strip()`.
- 263. Strip only the leading whitespaces from the left side of a string.
- 264. Strip only the trailing whitespaces from the right side of a string.
- 265. Replace all occurrences of a specific character in a string with another character.
- 266. Replace the word "apple" with "orange" in a given sentence.
- 267. Split a sentence into a collection of individual words based on spaces using `.split()`.
- 268. Join a collection of words into a single sentence string using a hyphen connector.
- 269. Check if a string starts with a specific prefix substring using `.startswith()`.
- 270. Check if a string ends with a specific suffix substring using `.endswith()`.
- 271. Find the starting index position of a substring within a text using `.find()`.
- 272. Count the exact number of times a substring appears inside a main string using `.count()`.
- 273. Check if a string is a valid palindrome (reads identical forwards and backwards).
- 274. Create a string made of the first two and last two characters of an input string.
- 275. Pad a string with asterisks on both sides so it is centered within a 20-character block.
- 276. Check if a substring exists within a string using the `in` keyword operator.
- 277. Check if a substring does *not* exist within a string using `not in`.
- 278. Repeat a string 5 times using the multiplication operator and print it.
- 279. Print the character code (ASCII value) of an input character.
- 280. Convert an ASCII code integer back into a character string.

- 281. Slice and print the first half of an input string (length divided by 2).
- 282. Slice and print the last half of an input string.
- 283. Remove all numerical digits from a string using a basic replacement process or filtering.
- 284. Remove all punctuation marks from an input text sentence.
- 285. Insert a space character between every letter of a given word.
- 286. Extract only the numerical digits from an alphanumeric string.
- 287. Form an email address by safely formatting name and domain string inputs into a single string.
- 288. Check if a string contains only uppercase letters.
- 289. Check if a string contains only lowercase letters.
- 290. Mask a string password by replacing all characters except the last two with asterisks.
- 291. Find the length of a string without using the built-in `len()` function.
- 292. Count the total number of sentences in a paragraph by counting periods (.
- 293. Swap the first and last characters of an input string.
- 294. Check if a string contains any special characters like \$, %, or *.
- 295. Trim a string to a maximum length of 10 characters and append "... " if it exceeds the limit.
- 296. Concatenate three different strings using the `.format()` placeholder strategy.
- 297. Check if a string is composed entirely of whitespace characters.
- 298. Find the index of the second occurrence of a specific character in a string.
- 299. Reverse the order of words in a sentence while keeping the words intact.
- 300. Interleave characters of two equal-length strings into a single combined string.

Chapter 7: Basic Lists & Arrays (301–350)

- 301. Create a list containing 5 integers and print the complete list.
 - 302. Access and print the third element from a list using its index.
 - 303. Access and print the very last element of a list using a negative index.
 - 304. Modify the value of the first element in an existing list.
 - 305. Append a new string element to the end of a list using `.append()`.
 - 306. Insert an element at index position 2 in a list using `.insert()`.
 - 307. Remove an element from a list by specifying its value using `.remove()`.
 - 308. Remove and print the last element of a list using `.pop()`.
 - 309. Remove an element at a specific index using the `del` keyword.
 - 310. Find the total length (number of elements) of a list using `len()`.
 - 311. Check if a specific value exists inside a list using the `in` operator.
 - 312. Count how many times a particular value appears in a list using `.count()`.
 - 313. Find the index position of the first occurrence of a specific value in a list.
 - 314. Clear all elements from a list so it becomes completely empty.
 - 315. Concatenate two lists together into a single combined list using the `+` operator.
 - 316. Repeat a list three times using the multiplication operator.
-

- 317. Slice a list to extract elements from index 1 to index 4.
- 318. Slice a list to reverse all its elements.
- 319. Create a list of numbers and find the maximum value using the built-in `max()` function.
- 320. Create a list of numbers and find the minimum value using the built-in `min()` function.
- 321. Calculate the sum of all elements in a numerical list using `sum()`.
- 322. Compute the average value of elements in a numerical list.
- 323. Write a `for` loop that iterates through a list and prints each element.
- 324. Sort a list of numbers in ascending order using the `.sort()` method.
- 325. Sort a list of numbers in descending order using `.sort(reverse=True)`.
- 326. Create a copy of a list using the `.copy()` method and verify they are separate objects.
- 327. Reverse the order of elements in a list in-place using `.reverse()`.
- 328. Create an empty list, populate it with 5 user-input numbers using a loop, and print it.
- 329. Given a list of integers, print only the even numbers present.
- 330. Given a list of integers, print only the odd numbers present.
- 331. Create a new list containing the squares of all numbers from an existing list.
- 332. Find the largest number in a list *without* using the built-in `max()` function (use a loop).
- 333. Find the smallest number in a list *without* using the built-in `min()` function (use a loop).
- 334. Count how many positive and negative numbers are present inside a list.
- 335. Multiply all elements in a numerical list together using a loop to find the total product.
- 336. Replace all occurrences of a specific value in a list with a new value.
- 337. Check if a list is empty by evaluating its truth value or length.
- 338. Merge two lists alternatingly element by element into a third list.
- 339. Separate a single list of mixed integers into two lists: one for evens and one for odds.
- 340. Convert a list of individual characters into a single unified string.
- 341. Find the second largest number in a list of integers using a basic loop strategy.
- 342. Remove all duplicate values from a list using basic checking loops (no sets).
- 343. Flatten a shallow nested list of lists (e.g., `[[1, 2], [3, 4]]`) into a single flat list.
- 344. Print elements of a list alongside their index numbers using a simple counter.
- 345. Check if all elements in a list are greater than a target threshold value.
- 346. Swap the first and last elements of a list dynamically.
- 347. Create a list of 10 zeros using list multiplication syntax.
- 348. Extract all strings from a list of mixed data types.
- 349. Slice out and print the middle element of an odd-length list.
- 350. Count how many items in a list are longer than 5 characters.

Chapter 8: Dictionaries & Key-Value Pairs (351–400)

- 351. Create a dictionary representing a book (title, author, year) and print it.
 - 352. Access the value of a specific key using square bracket syntax.
 - 353. Access the value of a specific key using the safely handled `.get()` method.
 - 354. Add a new key-value pair to an existing dictionary.
-

- 355. Update the value of an existing key in a dictionary.
 - 356. Remove a key-value pair from a dictionary using the `del` keyword.
 - 357. Remove a key-value pair using the `.pop()` method and print the removed value.
 - 358. Check if a specific key exists within a dictionary using the `in` operator.
 - 359. Get a collection of all keys present in a dictionary using `.keys()`.
 - 360. Get a collection of all values present in a dictionary using `.values()`.
 - 361. Get all key-value pairings as a list of tuples using `.items()`.
 - 362. Find the total number of items (key-value pairs) in a dictionary using `len()`.
 - 363. Clear all items from a dictionary to make it completely empty.
 - 364. Iterate through a dictionary using a `for` loop and print all keys.
 - 365. Iterate through a dictionary using a `for` loop and print all values.
 - 366. Iterate through a dictionary and print keys and values together formatted nicely.
 - 367. Create a dictionary from two separate lists (one for keys, one for values) using loops or `zip()`.
 - 368. Merge two separate dictionaries into a single combined dictionary.
 - 369. Create a copy of a dictionary using the `.copy()` method.
 - 370. Use the `.setDefault()` method to assign a default value to a key if it doesn't exist.
 - 371. Create a dictionary tracking student exam grades; calculate the average grade value.
 - 372. Find the key associated with the highest value in a numerical dictionary.
 - 373. Find the key associated with the lowest value in a numerical dictionary.
 - 374. Create a dictionary that maps numbers from 1 to 5 to their corresponding cube values.
 - 375. Count the frequency of each character in an input word and store the results in a dictionary.
 - 376. Count the frequency of each individual word in a sentence using a dictionary.
 - 377. Filter out and remove all items from a dictionary whose values are below a certain number.
 - 378. Check if a specific value exists inside a dictionary's values collection.
 - 379. Create a nested dictionary representing a classroom of students and their individual details.
 - 380. Access a deeply nested value inside a multi-layered dictionary structure.
 - 381. Update a specific field inside a nested dictionary structure.
 - 382. Invert a dictionary by swapping all its keys and values.
 - 383. Group a list of words into a dictionary indexed by their starting letter.
 - 384. Create a dictionary with predefined keys and initialize all values to zero.
 - 385. Compare two dictionaries to check if they are identical in content.
 - 386. Print a dictionary sorted alphabetically by its keys.
 - 387. Extract a subset of a dictionary based on a list of specified target keys.
 - 388. Update an existing dictionary with items from another dictionary using `.update()`.
 - 389. Create a simple item stock directory; decrease stock quantity when an item is purchased.
 - 390. Check if two different keys inside a dictionary hold identical values.
 - 391. Convert a dictionary into a list of plain strings containing "key: value".
 - 392. Create a dictionary representing a product catalog where values are lists of prices.
 - 393. Sum up all values of a dictionary that have a specific string prefix in their keys.
-

- 394. Delete a key from a dictionary safely without crashing if the key does not exist.
- 395. Convert a list of pairs (tuples) into a true Python dictionary object.
- 396. Find the length of the longest key string inside a dictionary.
- 397. Replace all blank values in a dictionary with a placeholder string "N/A".
- 398. Build a dictionary where values are booleans indicating whether the integer key is even.
- 399. Find the intersection of two dictionaries (keys shared by both with identical values).
- 400. Extract all unique values from a dictionary into a clean list.

Chapter 9: Tuples & Sets (401–450)

- 401. Create a tuple containing 5 unique elements and print it.
- 402. Access and print the second element of a tuple using index notation.
- 403. Try to change an element in a tuple and observe or handle the resulting `TypeError` exception.
- 404. Find the total length of a tuple using the `len()` function.
- 405. Check if a specific item exists inside a tuple using the `in` keyword.
- 406. Count the occurrences of a specific value inside a tuple using `.count()`.
- 407. Find the index position of a specific element in a tuple.
- 408. Unpack a tuple of 3 elements into three separate distinct variables.
- 409. Concatenate two tuples together into a new single tuple using the `+` operator.
- 410. Repeat a tuple 4 times using the multiplication operator.
- 411. Slice a tuple to extract a specific sub-tuple range.
- 412. Convert an existing tuple into a standard list.
- 413. Convert an existing list into an immutable tuple.
- 414. Find the maximum and minimum values inside a numerical tuple.
- 415. Create a tuple with only a single element (ensure the trailing comma is present).
- 416. Iterate through a tuple using a standard `for` loop and print items.
- 417. Create an empty set using the `set()` constructor function.
- 418. Add a new element to a set using the `.add()` method.
- 419. Add multiple elements to a set simultaneously using the `.update()` method.
- 420. Remove an element from a set using `.remove()` (raises error if missing).
- 421. Remove an element from a set safely using `.discard()` (no error if missing).
- 422. Pop an arbitrary element out of a set using the `.pop()` method.
- 423. Clear all elements from a set to empty it completely.
- 424. Check if an item exists inside a set using the `in` operator.
- 425. Find the mathematical union of two sets using the `|` operator or `.union()`.
- 426. Find the mathematical intersection of two sets using the `&` operator or `.intersection()`.
- 427. Find the difference between two sets using the `-` operator or `.difference()`.
- 428. Find the symmetric difference between two sets using the `^` operator.
- 429. Check if one set is a subset of another set using `.issubset()`.
- 430. Check if one set is a superset of another set using `.issuperset()`.

- 431. Check if two sets are completely disjoint (share no common elements).
- 432. Convert a list containing duplicate items into a set to eliminate duplicates, then print it.
- 433. Find the length of a set using the `len()` function.
- 434. Iterate through a set using a loop and print each item.
- 435. Create a frozen set using `frozenset()` and explain its behavior compared to a normal set.
- 436. Extract elements that are unique to the first set and not present in the second set.
- 437. Convert a string sentence into a set of its individual unique words.
- 438. Convert a tuple into a set.
- 439. Create a set containing the squares of numbers from 1 to 5.
- 440. Check if a set is completely empty.
- 441. Find the sum of all elements in a numerical set.
- 442. Unpack elements of a tuple using the wildcard asterisk syntax (`*rest`).
- 443. Test if a specific value is absent from a set using `not in`.
- 444. Filter out any words from a set that are shorter than 3 characters long.
- 445. Create a dictionary where keys are immutable tuples representing coordinate points.
- 446. Compare two sets to check if they are identical in content regardless of order.
- 447. Create a set of uppercase alphabet characters extracted from an input string.
- 448. Determine if a list contains any duplicate values by comparing list length to set length.
- 449. Merge three separate sets into a single complete set.
- 450. Print the elements of a set sorted in ascending order (returns a sorted list).

Chapter 10: User-Defined Functions (451–500)

- 451. Define a function named `greet()` that prints "Hello, World!". Call the function.
- 452. Define a function that takes a name string as a parameter and prints a personalized greeting.
- 453. Write a function `add(a, b)` that returns the sum of two numbers.
- 454. Write a function `square(x)` that returns the square of an input number.
- 455. Write a function `is_even(num)` that returns `True` if a number is even, and `False` otherwise.
- 456. Create a function that calculates the area of a rectangle and returns it.
- 457. Create a function with a default parameter value (e.g., `greet(name="Guest")`).
- 458. Write a function that accepts two arguments and returns the larger one.
- 459. Create a function `factorial(n)` that computes and returns the factorial of `n` using a loop.
- 460. Write a function that takes a string and returns its total character length.
- 461. Define a function that takes a string and returns the same string completely reversed.
- 462. Create a function that accepts a list of numbers and returns the sum of all elements.
- 463. Write a function that counts and returns the number of vowels present inside an input string.
- 464. Write a function that takes a temperature in Celsius and returns its Fahrenheit equivalent.

465. Create a function `is_prime(n)` that returns `True` if a number is prime, otherwise `False`.
 466. Write a function that accepts a list of integers and returns a new list containing only the even numbers.
 467. Create a function that calculates simple interest and returns the calculated value.
 468. Write a function that checks if an input string is a palindrome.
 469. Define a function that accepts any number of positional arguments using `*args` and prints them.
 470. Define a function that accepts arbitrary keyword arguments using `kwargs` and loops through them.
 471. Create a function that takes a sentence and returns the count of individual words inside it.
 472. Write a function that returns multiple values simultaneously using a tuple format.
 473. Create a function that checks if a specific key exists within a given dictionary.
 474. Write a function that takes a list of strings and returns the longest string in the list.
 475. Create a function that takes a list of numbers and returns their average.
 476. Write a function that converts an amount in centimeters to meters and returns it.
 477. Define a small lambda function that adds 10 to any input argument.
 478. Define a lambda function that multiplies two numbers together.
 479. Pass a lambda function into a built-in filter statement or use it directly as a shortcut function.
 480. Demonstrate variable scope by defining a local variable inside a function and trying to print it outside.
 481. Use the `global` keyword inside a function to modify a variable defined at the top level.
 482. Write a function that takes a radius and returns the circumference of a circle.
 483. Create a function that accepts a list of numbers and returns a new list with all duplicates removed.
 484. Create a function that formats a float number into a clean percentage string.
 485. Write a function that checks if an age qualifies a person for a senior discount.
 486. Create a function that checks if a string contains any numerical digits.
 487. Write a function that takes a list of numbers and returns the smallest value found.
 488. Define a function that accepts a base and an exponent and returns the calculated power result.
 489. Create a validator function that returns `True` if a username string is between 5 and 12 characters long.
 490. Write a function that simulates a coin flip, returning "Heads" or "Tails" based on a condition.
 491. Write a function that takes a list of items and returns a dictionary counting item frequencies.
 492. Create a function that strips all whitespaces and converts an entire text input to title case.
 493. Write a function that accepts a number of days and returns the equivalent number of hours.
 494. Create a function that checks if three input side lengths can form a valid triangle.
 495. Write a function that takes a dictionary and returns a list containing only its unique values.
-

- 496. Create a function that accepts a text file name string and checks if it ends with a `.txt` extension.
- 497. Define a function that checks if a target integer is a multiple of an input factor.
- 498. Write a function that returns the absolute mathematical difference between two numbers.
- 499. Create a function that takes an integer and returns a string of that number padded with leading zeros.
- 500. Write an empty placeholder function using the `pass` keyword to map out an architectural outline.

Chapter 11: File Input & Output Basics (501–550)

- 501. Open and create a new text file named `test.txt` in write mode (`'w'`), then close it.
- 502. Write a single line of text `"Hello, File World!"` into a text file.
- 503. Use the safe `with open(...)` context manager to automatically open and close a file.
- 504. Open an existing text file in read mode (`'r'`) and print its entire content string.
- 505. Write three separate lines of text into a text file using a single write action or a loop.
- 506. Read an entire text file line by line using a standard `for` loop over the file object.
- 507. Read all lines of a text file into a Python list of strings using the `.readlines()` method.
- 508. Open a file in append mode (`'a'`) and add a new line of text to the bottom without erasing existing data.
- 509. Read only the first line of a text file using the `.readline()` method.
- 510. Read the first 10 characters of a text file by passing an argument to `.read()`.
- 511. Count the total number of lines present in an input text file.
- 512. Count the total number of individual words inside a text file.
- 513. Count the total number of characters present in a text file.
- 514. Copy the entire contents of a file named `source.txt` over into a new file named `destination.txt`.
- 515. Ask a user for a string line input and append that exact input directly to a log file.
- 516. Open a file and check if it is explicitly readable or writable using `.readable()` and `.writable()`.
- 517. Read a text file and print only lines that start with the prefix string `"ERROR:"`.
- 518. Search a text file for a target word; print a message stating whether the word was found.
- 519. Write a list of string elements into a file cleanly using the `.writelines()` method.
- 520. Create a blank file named `empty.txt` by opening it in write mode and closing it immediately.
- 521. Read a text file and print every line with its corresponding line number prefixed at the start.
- 522. Read a text file and output its content string completely converted into uppercase letters.
- 523. Find the longest line inside a text file and print its total length.
- 524. Read a file containing a list of numbers (one per line) and calculate their total sum.
- 525. Read a file containing numbers and calculate the average value of those numbers.
- 526. Overwrite an existing text file completely with a blank string to truncate its contents.

- 527. Read a text file and write its contents backward character by character into a separate output file.
- 528. Count how many times a specific target character appears inside an external text file.
- 529. Create a program that combines the lines of two text files alternately into a third output file.
- 530. Read a file and replace every occurrence of spaces with a hyphen character inside the file stream.
- 531. Strip all trailing newline characters (`\n`) from lines while reading a file into a clean list.
- 532. Write a program that verifies if a file exists before attempting to open it (use basic handlers).
- 533. Write a dictionary into a text file formatted manually as standard "key: value" text lines.
- 534. Read a structured configuration file line by line and parse keys and values back into a functional dictionary.
- 535. Filter a file to copy only lines that are longer than 20 characters over to a separate destination file.
- 536. Print the current cursor position within an active file stream using the `.tell()` method.
- 537. Move the active file stream cursor position back to the absolute beginning of a file using `.seek(0)`.
- 538. Write a multi-line poem into a file and read it back to confirm formatting blocks match.
- 539. Read a comma-separated file string line and split its values into a clean Python list object.
- 540. Read a file and count how many lines are completely empty.
- 541. Append a timestamp string entry to a log file.
- 542. Read a text file and print only the final line of the file.
- 543. Write numbers from 1 to 100 on separate lines inside a generated text file.
- 544. Read a text file containing names and print them out sorted alphabetically.
- 545. Read a file and filter out all numerical digits, printing only the alphabetic characters remaining.
- 546. Open a file in exclusive creation mode (`'x'`) and observe what happens if the file already exists.
- 547. Write a program that clears out lines from a text file that contain a specific banned keyword.
- 548. Read a paragraph file and calculate the average word length across the file.
- 549. Write a list of floating-point numbers into a text file rounded strictly to 1 decimal place.
- 550. Create an automation script that appends a dashed separator line `---` to the end of a file.

Chapter 12: Basic Modules & Standard Utilities (551–600)

- 551. Import the standard math module and calculate the square root of an input number using `math.sqrt()`.
 - 552. Use `math.ceil()` to round a floating-point number upward to its nearest integer.
 - 553. Use `math.floor()` to round a floating-point number downward to its nearest integer.
-

- 554. Access and print the precise value of the mathematical constant Pi (π) from the `math` module.
 - 555. Calculate the factorial of a number using the built-in `math.factorial()` function.
 - 556. Import the `random` module and generate a random floating-point number between 0.0 and 1.0.
 - 557. Generate a random integer within a range from 1 to 100 inclusive using `random.randint()`.
 - 558. Generate a random integer from a range sequence with a specific step value using `random.randrange()`.
 - 559. Use `random.choice()` to pick a random string element out of a list of options.
 - 560. Shuffle the order of elements in a list in-place using the `random.shuffle()` function.
 - 561. Import the `datetime` module and print the current date and time.
 - 562. Extract and print only the current year out of a `datetime.now()` object.
 - 563. Extract and print the current day of the week from a date object.
 - 564. Format the current date into a clean string using `.strftime()` (e.g., "Saturday, May 30, 2026").
 - 565. Calculate the absolute time difference between two dates using a `timedelta` object.
 - 566. Use the `os` module to print your current active working directory path string.
 - 567. Use the `sys` module to print the current version of the Python interpreter running.
 - 568. Print the total number of command-line arguments passed into a script using `sys.argv`.
 - 569. Print the absolute value of a negative number using `math.fabs()`.
 - 570. Use `math.pow()` to calculate the result of raising a base number to an exponent power.
 - 571. Calculate the greatest common divisor of two integers using `math.gcd()`.
 - 572. Check if a number is NaN (Not a Number) using the `math.isnan()` verification utility.
 - 573. Randomly select a sample subset of 3 unique items from a list of 10 items using `random.sample()`.
 - 574. Simulate rolling a standard 6-sided die by generating a random number between 1 and 6.
 - 575. Simulate flipping a fair coin using a random choice generator.
 - 576. Print a formatted calendar page for a specific month and year using the `calendar` module.
 - 577. Check if a specific target year is a leap year using the utility function `calendar.isleap()`.
 - 578. Import the `time` module and pause the execution of a script for exactly 2 seconds using `time.sleep()`.
 - 579. Measure the exact execution time of a block of basic code using `time.time()` performance markers.
 - 580. Use the `string` module to print a compiled string containing all lowercase and uppercase alphabet characters.
 - 581. Print a string containing all standard punctuation marks using `string.punctuation`.
 - 582. Use `random.choices()` to pick multiple random items from a list with replacement allowed.
 - 583. Calculate the natural logarithm of a number using `math.log()`.
 - 584. Convert an angle from degrees to radians using `math.radians()`.
 - 585. Convert an angle from radians to degrees using `math.degrees()`.
-

586. Use the `json` module to parse a valid JSON string structure into a functional Python dictionary.
587. Convert a standard Python dictionary object into a formatted JSON string using `json.dumps()`.
588. Import a specific function from a module directly using the `from module import function` syntax.
589. Import a module and assign it a custom shorthand namespace alias using the `import module as alias` syntax.
590. List all built-in functions, variables, and properties available within a module using the `dir()` utility.
591. Generate a secure random password string by picking characters out of a string module library.
592. Calculate the sine of an angle given in radians using `math.sin()`.
593. Calculate the cosine of an angle given in radians using `math.cos()`.
594. Check your operating system platform name using the `sys.platform` property string.
595. Use `math.fsum()` to calculate an accurate running total sum of a list of floating-point values.
596. Use the `keyword` module to print a comprehensive list of all reserved keywords in Python.
597. Check if a specific string is a reserved Python keyword using `keyword.iskeyword()`.
598. Print the current local time in epoch seconds format.
599. Print the value of a number raised to a power and then modulo-ed using the 3-argument built-in `pow(base, exp, mod)`.
600. Create a custom basic calculation utility module in a separate file, import it into a main script, and use its functions.